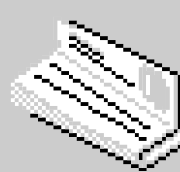
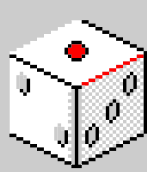
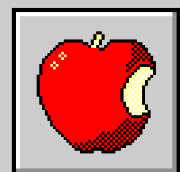
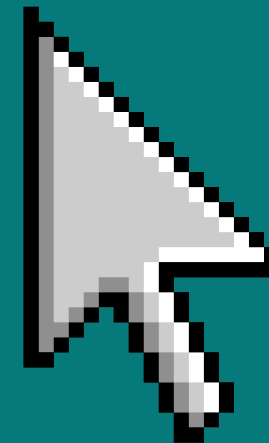


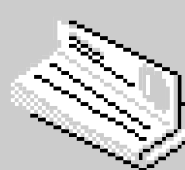
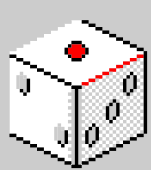
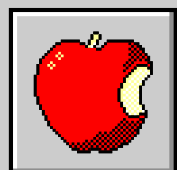
Proyecto PROGRAMACION 2026-A



11:11PM

HeidLine

"Ordena. Descubre.
Disfruta."

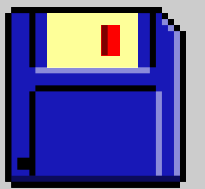
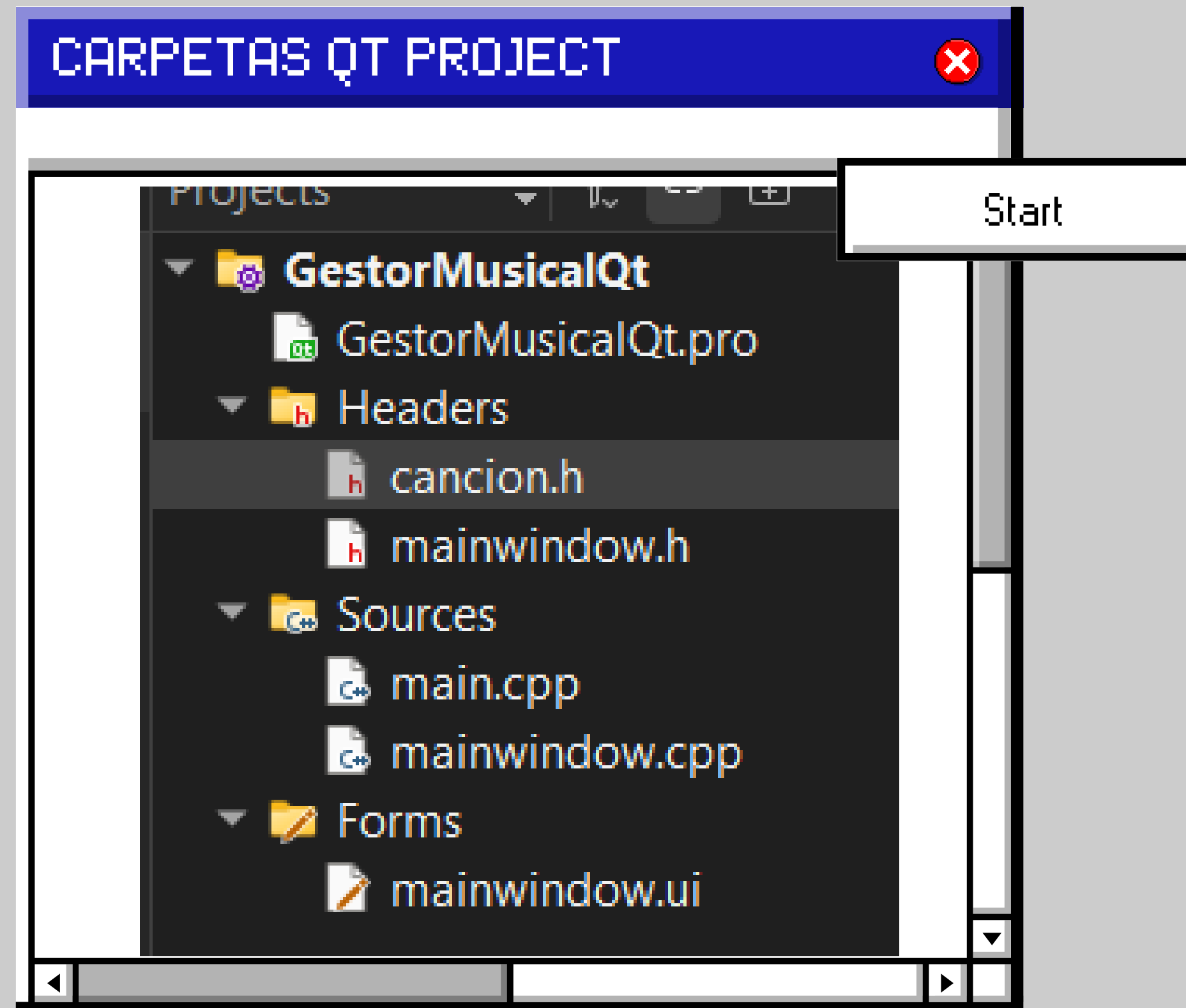


[Back to Agenda Page](#)

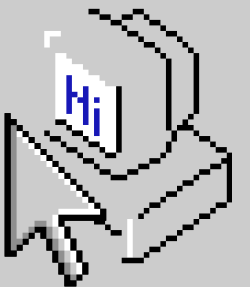


Servicio??

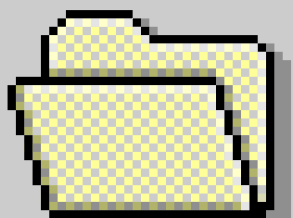
"Separamos la interfaz visual (.ui), las declaraciones (.h) y la lógica del programa (.cpp) para mantener el código organizado — es una práctica estándar en proyectos de Qt."



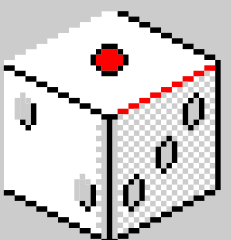
Qt Creator



HeidLine



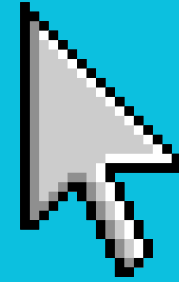
Files



RedPanda

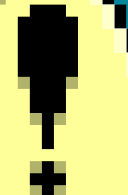
FUNCIONALIDAD

HeidLine



canciones.h

Declara el struct Cancion (los 7 campos: código, título, artista, género, año, duración, favorita) y las funciones para guardar/leer una canción del archivo de texto



```
#ifndef CANCION_H
#define CANCION_H

#include <iostream>
#include <string>
using namespace std;

struct Cancion {
    string codigo;
    string titulo;
    string artista;
    string genero;
    int anioLanzamiento = 0;
    double duracionMinutos = 0.0;
    bool favorita = false;
};

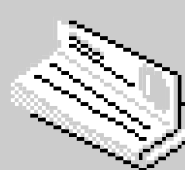
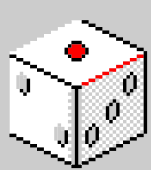
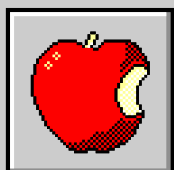
// Guarda una cancion en una linea del archivo, separada por ";"
inline ostream& operator<<(ostream& salida, const Cancion& c) {
    salida << c.codigo << ";"
        << c.titulo << ";"
        << c.artista << ";"
        << c.genero << ";"
        << c.anioLanzamiento << ";"
        << c.duracionMinutos << ";"
        << c.favorita;
    return salida;
}

// Lee una cancion desde una linea del archivo
inline istream& operator>>(istream& entrada, Cancion& c) {
    string anio, duracion, favorita;

    if (getline(entrada, c.codigo, ';') &&
        getline(entrada, c.titulo, ';') &&
        getline(entrada, c.artista, ';') &&
        getline(entrada, c.genero, ';') &&
        getline(entrada, anio, ';') &&
        getline(entrada, duracion, ';') &&
        getline(entrada, favorita))
    {
        c.anioLanzamiento = stoi(anio);
        c.duracionMinutos = stod(duracion);
        c.favorita = stoi(favorita);
    }

    return entrada;
}

#endif
```



BACK :V

```

#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <vector>
#include <string>
#include "cancion.h"

QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE

class MainWindow : public QMainWindow {
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private slots:
    //CRUD Numero 1: Canciones
    void onCrearCancion();
    void onActualizarCancion();
    void onEliminarCancion();
    void onLimpiarFormulario();
    void onFilaCancionSeleccionada(int fila, int columna);

    // CRUD Numero 2: Categorías
    //Navegacion entre las pestañas
    void onIrACategorías();
    void onCrearCategoría();
    void onActualizarCategoría();
    void onEliminarCategoría();
    void onUsarCategoríaSeleccionada();

```

```

private:
    Ui::MainWindow *ui;

    //Datos en memoria
    std::vector<Cancion> canciones;
    std::vector<std::string> categorias;
    const std::string archivoCanciones = "canciones.txt";
    const std::string archivoCategorias = "categorias.txt";

    //Persistencia
    void cargarCanciones();
    void guardarCanciones();
    int buscarCancionPorCodigo(const std::string &codigo);

    void cargarCategorias();
    void guardarCategorias();
    int buscarCategoríaPorNombre(const std::string &nombre);

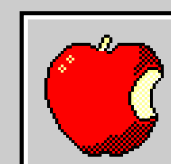
    // UI
    void configurarTabla();
    void refrescarTablaCanciones();
    void refrescarListaCategorias();
    void limpiarFormularioCancion();
};

#endif

```

mainwindow.h

Declara la clase MainWindow:
qué botones/campos tiene, y la
"firma" de cada función (crear,
actualizar, eliminar, etc.) sin
mostrar el código de adentro



[Back to Agenda Page](#)

Sources (archivos .cpp)

Son los archivos de implementación — aquí sí está el código real, la lógica que hace que las cosas funcionen.

main.cpp

El punto de entrada del programa.
Es lo primero que se ejecuta: crea la aplicación Qt y muestra la ventana principal



```
main.cpp
#include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[]) {
    QApplication a(argc, argv);
    MainWindow w;
    w.show();
    return a.exec();
}
```

[Back to Agenda Page](#)



mainwindow.cpp

Toda la lógica real: qué pasa cuando le das clic a "Crear", cómo se guarda en el .txt, cómo se cambia de pestaña, etc.



[Back to Agenda Page](#)

```
mainwindow.cpp
#include "mainwindow.h"
#include "ui_mainwindow.h"

#include <QMessageBox>
#include <QHeaderView>
#include <fstream>
using namespace std;

MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent), ui(new Ui::MainWindow) {
    ui->setupUi(this);

    configurarTabla();

    cargarCanciones();
    cargarCategorias();

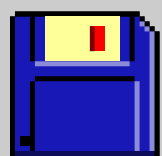
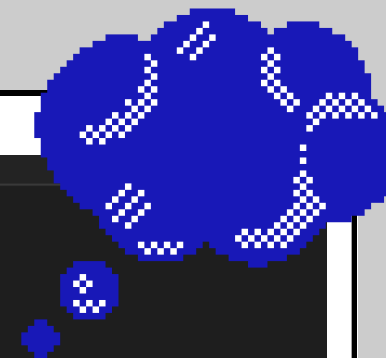
    refrescarTablaCanciones();
    refrescarListaCategorias();

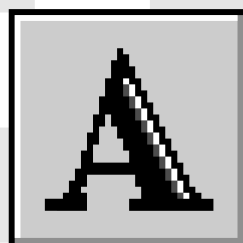
    connect(ui->btnElegirCategoria, &QPushButton::clicked, this, &MainWindow::onIrACategorias);
    connect(ui->btnCrearCancion, &QPushButton::clicked, this, &MainWindow::onCrearCancion);
    connect(ui->btnActualizarCancion, &QPushButton::clicked, this, &MainWindow::onActualizarCancion);
    connect(ui->btnEliminarCancion, &QPushButton::clicked, this, &MainWindow::onEliminarCancion);
    connect(ui->btnLimpiarCancion, &QPushButton::clicked, this, &MainWindow::onLimpiarFormulario);
    connect(ui->tablaCanciones, &QTableWidget::cellClicked, this, &MainWindow::onFilaCancionSeleccionada);

    connect(ui->btnCrearCategoria, &QPushButton::clicked, this, &MainWindow::onCrearCategoria);
    connect(ui->btnActualizarCategoria, &QPushButton::clicked, this, &MainWindow::onActualizarCategoria);
    connect(ui->btnEliminarCategoria, &QPushButton::clicked, this, &MainWindow::onEliminarCategoria);
    connect(ui->btnUsarCategoria, &QPushButton::clicked, this, &MainWindow::onUsarCategoriaSeleccionada);
}

MainWindow::~MainWindow() {
    delete ui;
}

void MainWindow::configurarTabla() {
    ui->tablaCanciones->horizontalHeader()->setStretchLastSection(true);
}
```





PRESS START



mainwindow.ui

Type Here

Canciones Categorías

HeidLine 

Código:

Título:

Artista:

Genero:

Año:

Duración (min):

☐ Es favorita

Canciones registradas:

Código	Título	Artista	Genero	Año	Duración	Fa

mainwindow.ui

El diseño visual de la ventana (botones, campos de texto, tabla, pestañas), guardado como XML. Es lo que editas arrastrando cosas en el Modo Diseño. Qt lo convierte automáticamente a código C++ (ui_mainwindow.h) cuando compilas — por eso en mainwindow.cpp puedes usar ui->leCodigo, ui->btnCrear, etc.





Thank you!



Whiteboard Page

Additional Information



Carrera Marlon
+593 97 870
0701



Villa Roddik
+593 98 359
6474



Gaona Alfredo
+593 98 094
4596

